

LAPORAN OS TUGAS 5 (NIM GANJIL)
SYSTEM BOOKING HOTEL



Dosen Pengampu :
Triawan Adi Cahyanto M.kom

Disusun Oleh :
Hafabi Arrohim (2410651013)
Kelas A jumat

PROGRAM STUDY TEKNIK INFORMATIKA FAKULTAS
TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER TAHUN 2025

DAFTAR ISI

DAFTAR ISI.....	i
1. Soal Pemograman	1
1.1 Analisis Masalah.....	1
1.2 Implementasi	2
1.3 Testing dan output.....	9
1.4 Analisis Hasil.....	11
2. Kesimpulan.....	12
3. Referensi	13

1. Soal Pemograman

Buat simulasi sistem booking hotel:

- Hotel memiliki 5 kamar.
- Ada 10 customer (thread) yang mencoba booking bersamaan.
- Customer mencoba booking (random 1-3 detik initial), menginap 5 detik jika berhasil, cek-out melepas kamar.
- Customer maksimal mencoba 3 kali.
- Gunakan semaphore untuk mengatur jumlah kamar, dan mutex untuk menjaga kelengkapan output/operasi.

1.1 Analisis Masalah

A. Pemahaman anda tentang soal

Tugas ini meminta untuk membuat simulasi sistem booking hotel menggunakan konsep multithreading dan sinkronisasi. Dalam simulasi ini terdapat 10 thread (customer) yang berusaha memesan kamar di hotel yang hanya memiliki 5 kamar. Setiap customer akan mencoba memesan kamar secara bersamaan, dan jika kamar penuh maka ia akan menunggu lalu mencoba lagi hingga maksimal tiga kali. Jika berhasil, customer “menginap” selama beberapa detik sebelum melakukan check-out dan melepaskan kamar yang telah digunakan.

Tujuannya adalah untuk menunjukkan bagaimana sistem operasi mengatur beberapa thread yang berjalan secara paralel tanpa saling bertabrakan dalam mengakses sumber daya bersama (kamar hotel).

B. Identifikasi masalah sinkronisasi

Masalah utama dalam simulasi ini adalah adanya sumber daya terbatas (5 kamar) yang diakses bersamaan oleh banyak thread (10 customer). Jika tidak dilakukan sinkronisasi, bisa terjadi beberapa masalah seperti:

- Race Condition dua atau lebih thread mengakses dan memesan kamar yang sama pada waktu bersamaan.

- Data Inconsistency data kamar bisa menjadi salah (dua customer dianggap menempati kamar yang sama).
- Output Berantakan beberapa thread mencetak output ke terminal bersamaan tanpa urutan yang jelas.

Untuk menghindari masalah tersebut, dibutuhkan mekanisme sinkronisasi agar akses ke data kamar dilakukan secara tertib dan aman.

C. Desain Solusi

Semua thread dijalankan bersamaan (10 customer).

Thread mencoba booking:

Jika semaphore masih > 0 (ada kamar kosong), thread dapat mengunci mutex, memilih kamar, lalu menginap.

Jika semaphore = 0 (penuh), thread gagal booking, menunggu 2 detik, dan mencoba lagi (maksimal 3 kali).

Setelah selesai menginap, thread melakukan check-out:

Mengunci mutex, menandai kamar sebagai kosong, lalu `sem_post()` agar kamar bisa digunakan lagi.

Dengan rancangan ini, sistem berjalan tertib, aman dari race condition, dan output tetap teratur meskipun semua customer berjalan bersamaan.

1.2 Implementasi

A. Penjelasan Algoritma

Program dibuat menggunakan bahasa C di Linux dengan library `pthread.h` dan `semaphore.h`. Struktur utama program terdiri dari:

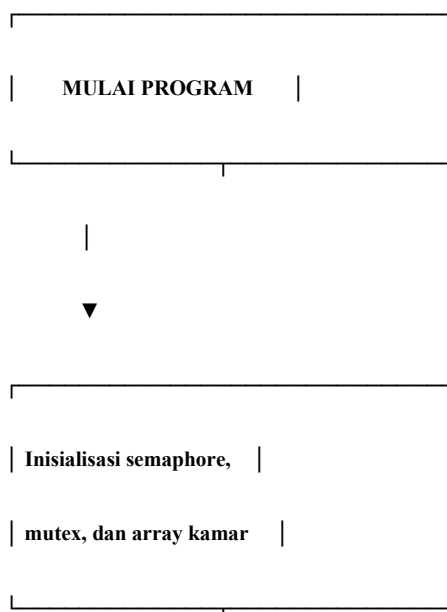
- Semaphore (`rooms_sem`) untuk membatasi jumlah kamar yang dapat diakses secara bersamaan.

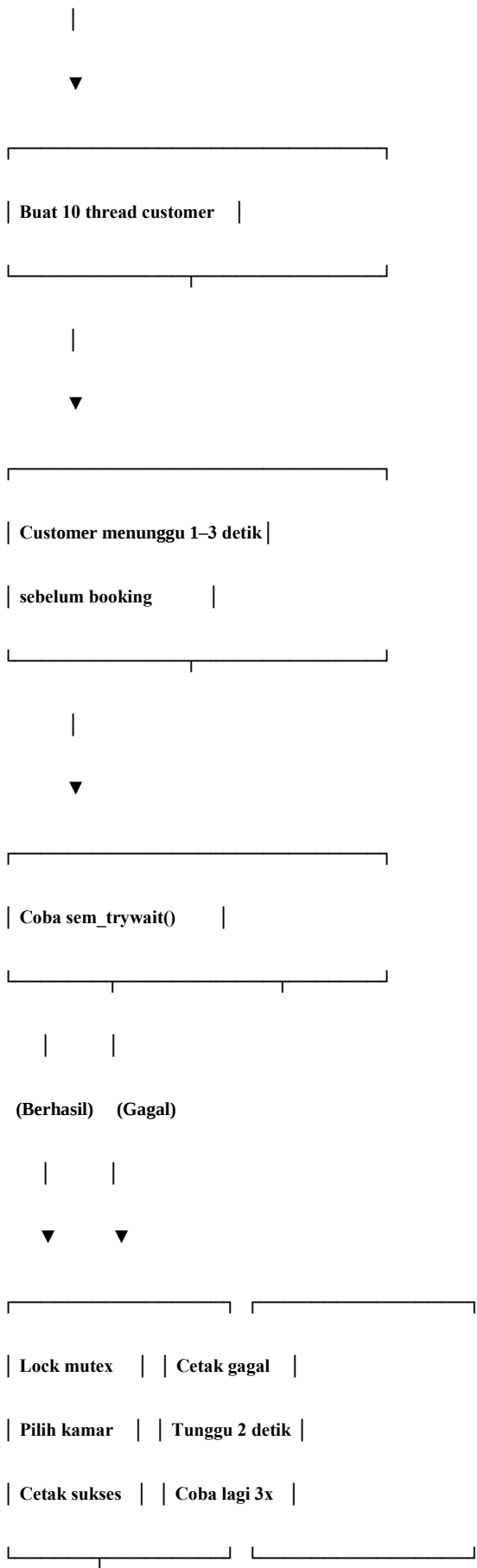
- Mutex (`rooms_mutex`) untuk melindungi data kamar (`rooms[]`) dan mencegah output tumpang tindih.
- Array `rooms[5]` untuk menyimpan status kamar, diisi dengan ID customer yang menempatinnya, atau -1 jika kosong.
- Thread (`pthread_t`) untuk mewakili customer yang berusaha melakukan booking.

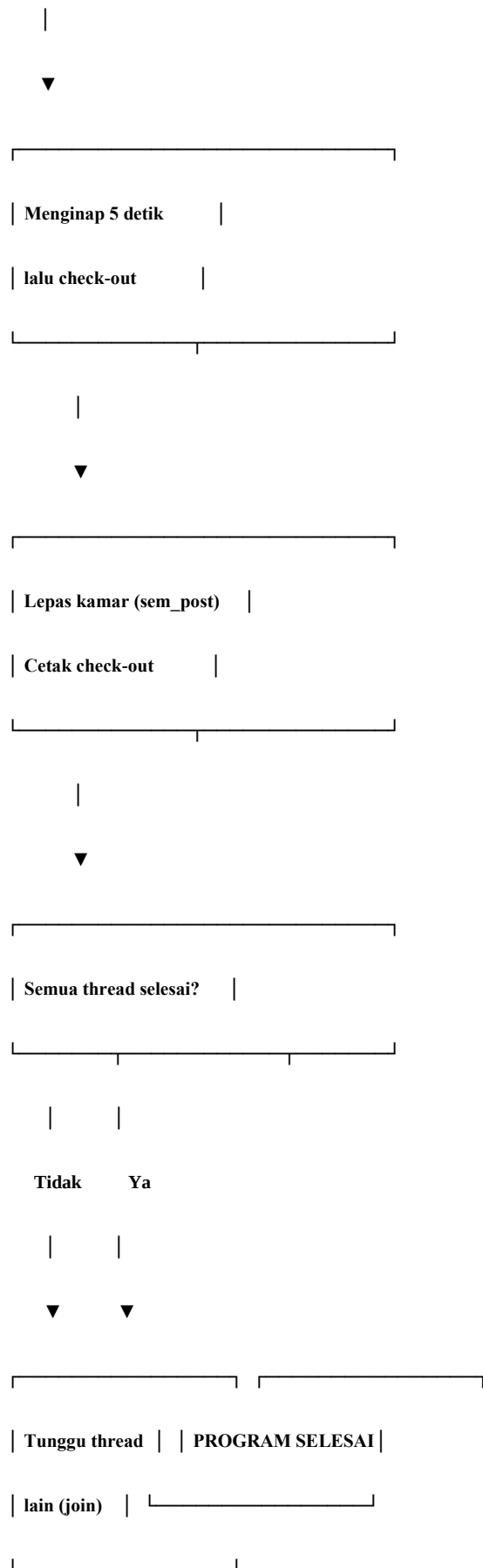
B. Alur Program

- Inisialisasi semaphore dengan nilai awal = 5 (jumlah kamar).
- Inisialisasi mutex untuk penguncian data bersama.
- Jalankan 10 thread yang mewakili customer.
- Masing-masing thread akan:
 - Menunggu waktu acak 1–3 detik sebelum mencoba booking.
 - Melakukan `sem_trywait()` untuk mengecek ketersediaan kamar.
 - Jika berhasil → masuk critical section dengan `pthread_mutex_lock()` untuk memilih kamar kosong.
 - Jika gagal → menunggu 2 detik lalu mencoba lagi hingga 3 kali.
 - Setelah berhasil booking, thread akan “menginap” selama 5 detik.
 - Kemudian *check-out*, mengosongkan kamar, dan melakukan `sem_post()` agar kamar bisa digunakan kembali.

C. Flowchat Sederhana







D. Potongan kode penting dengan penjelasan

```
// inisialisasi rooms array (-1 = kosong)
```

```
for (int i = 0; i < TOTAL_ROOMS; i++) rooms[i] = -1;
```

```
// inisialisasi semaphore dan mutex
```

```
sem_init(&rooms_sem, 0, TOTAL_ROOMS); // semaphore dengan nilai awal = jumlah kamar
```

```
pthread_mutex_init(&rooms_mutex, NULL); // mutex untuk proteksi struktur data & output
```

penjelasan :

- `rooms[]` menyimpan status tiap kamar.
- `sem_init(&rooms_sem, 0, TOTAL_ROOMS)` memberikan *counter* awal = jumlah kamar; setiap booking harus mendapat permit (mengurangi counter).
- `rooms_mutex` digunakan untuk melindungi operasi baca/tulis `rooms[]` dan semua `printf` agar output terminal tidak tercampur.

```
if (sem_trywait(&rooms_sem) == 0) {
```

```
    // mendapat permit -> masuk critical section untuk memilih kamar
```

```
    pthread_mutex_lock(&rooms_mutex);
```

```
    int room = try_book_room(id); // cari index kamar kosong, tandai dengan id
```

```
    // ... print berhasil & hitung kamar tersisa ...
```

```
    pthread_mutex_unlock(&rooms_mutex);
```



```

// menginap...

sleep(5);

// check-out

pthread_mutex_lock(&rooms_mutex);

release_room(room); // rooms[room] = -1

sem_post(&rooms_sem); // kembalikan permit semaphore

pthread_mutex_unlock(&rooms_mutex);

} else {

// jika sem_trywait gagal -> tidak ada kamar sekarang

// print gagal, tunggu 2s, coba lagi (hingga MAX_TRIES)

}

```

Penjelasan :

- `sem_trywait()` mencoba mengambil satu permit tanpa memblokir; bila gagal, thread dapat melakukan retry sesuai spesifikasi tugas (menunggu 2 detik lalu mencoba lagi).
- Setelah mendapat permit, lakukan `pthread_mutex_lock()` sebelum mengubah `rooms[]` agar tidak terjadi *race condition* saat dua thread memilih kamar yang sama.
- Saat check-out, kita kembalikan `rooms[room] = -1` dan `sem_post()` untuk menambah kembali permit semaphore (menandakan ada kamar tersedia).

```

int try_book_room(int cust_id) {

    int found = -1;

    for (int i = 0; i < TOTAL_ROOMS; i++) {

        if (rooms[i] == -1) {

```

```

        rooms[i] = cust_id;

        found = i;

        break;
    }

}

return found;
}

void release_room(int room_idx) {

    rooms[room_idx] = -1;

}

```

Penjelasan :

- try_book_room melakukan pencarian linear untuk kamar kosong dan menandainya. Harus dipanggil hanya saat sudah memegang rooms_mutex.
- release_room mengosongkan kamar saat check-out.

E. Mengapa Memilih Mekanisme Tertentu

- Semaphore digunakan untuk membatasi jumlah kamar yang bisa dipakai secara bersamaan (maksimal 5). Mekanisme ini cocok untuk mengatur resource sharing dengan kapasitas terbatas.
- Mutex digunakan untuk melindungi bagian critical section seperti akses ke array rooms[] dan proses printf, agar tidak terjadi race condition atau output tumpang tindih.
- Kombinasi keduanya memastikan program berjalan tertib, aman, dan bebas konflik meskipun banyak thread berjalan bersamaan.

1.3 Testing dan output

```
// File: hotel_booking.c
// Compile: gcc hotel_booking.c -o hotel_booking -pthread
// Run: ./hotel_booking

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <string.h>

#define TOTAL_ROOMS 5
#define TOTAL_CUSTOMERS 10
#define MAX_TRIES 3

// Shared resources
int rooms[TOTAL_ROOMS]; // -1 = free, else customer id
sem_t rooms_sem; // counts available rooms
pthread_mutex_t rooms_mutex; // protect rooms array and prints

// Helper: current time string
void timestr(char *buf, size_t n) {
    time_t t = time(NULL);
    struct tm tm = *localtime(&t);
    strftime(buf, n, "%H:%M:%S", &tm);
}

// Try to book a room. returns room index ≥ 0 if success, -1 if none.
int try_book_room(int cust_id) {
    int found = -1;
    // find a free room
    for (int i = 0; i < TOTAL_ROOMS; i++) {
        if (rooms[i] == -1) {
            rooms[i] = cust_id;
            // -----
            sleep(5);

            // check-out
            pthread_mutex_lock(&rooms_mutex);
            release_room(room);
            // release semaphore slot
            sem_post(&rooms_sem);
            timestr(tbuf, sizeof(tbuf));
            // recompute rooms left
            int now_left = 0;
            for (int i = 0; i < TOTAL_ROOMS; i++) if (rooms[i] == -1) now_left++;
            printf("[%s] █ Customer-%d check-out (Kamar %d) - (%d kamar tersisa)\n", tbuf, id, room + 1, now_left);
            pthread_mutex_unlock(&rooms_mutex);

            // finished (successful booking)
            break;
        } else {
            // This shouldn't usually happen because sem granted, but handle it
            pthread_mutex_unlock(&rooms_mutex);
            // release the permit because we couldn't find a room
            sem_post(&rooms_sem);
        }
    }
    // sem_trywait failed → no room right now
    pthread_mutex_lock(&rooms_mutex);
    timestr(tbuf, sizeof(tbuf));
    printf("[%s] ✖ Customer-%d gagal booking (kamar penuh), menunggu da\n", tbuf, id);
    pthread_mutex_unlock(&rooms_mutex);
    if (attempt < MAX_TRIES) {
        sleep(2); // wait 2s before retry
    } else {
        pthread_mutex_lock(&rooms_mutex);
        timestr(tbuf, sizeof(tbuf));
        printf("[%s] ✖ Customer-%d menyerah setelah %d percobaan\n", tbuf, id, attempt);
        pthread_mutex_unlock(&rooms_mutex);
    }
}

void release_room(int room_idx) {
    rooms[room_idx] = -1;
}

void* customer_thread(void* arg) {
    int id = *(int*)arg;
    char tbuf[16];

    // random delay before first attempt (1-3s)
    int initial_wait = (rand() % 3) + 1;
    sleep(initial_wait);

    for (int attempt = 1; attempt ≤ MAX_TRIES; attempt++) {
        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&rooms_mutex);
        printf("[%s] ★ Customer-%d mencoba booking... (percobaan %d)\n", tbuf, id, attempt);
        pthread_mutex_unlock(&rooms_mutex);

        // Try to get a slot (decrement semaphore). If no room, sem_trywait couldn't
        // but we want to block only if willing to wait; here we use sem_trywait
        if (sem_trywait(&rooms_sem) == 0) {
            // got a "permit" - now lock and choose a room index
            pthread_mutex_lock(&rooms_mutex);
            int room = try_book_room(id);
            if (room ≥ 0) {
                timestr(tbuf, sizeof(tbuf));
                int rooms_left = 0;
                // count rooms left for reporting
                for (int i = 0; i < TOTAL_ROOMS; i++) if (rooms[i] == -1) rooms_left++;
                printf("[%s] ✓ Customer-%d berhasil booking! (Kamar %d) - (%d kamar tersisa)\n", tbuf, id, room + 1, rooms_left);
                pthread_mutex_unlock(&rooms_mutex);

                // stay 5s
                sleep(5);
            } else {
                // no room available, release permit and try again
                sem_post(&rooms_sem);
            }
        } else {
            // no permit, try again later
            sleep(1);
        }
    }
}

int main() {
    srand(time(NULL));
    // init rooms
    for (int i = 0; i < TOTAL_ROOMS; i++) rooms[i] = -1;

    // init sync primitives
    sem_init(&rooms_sem, 0, TOTAL_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);

    pthread_t customers[TOTAL_CUSTOMERS];
    int ids[TOTAL_CUSTOMERS];

    printf("≡ SISTEM BOOKING HOTEL ≡\n");
    printf("Total Kamar: %d\n", TOTAL_ROOMS);

    // create customer threads almost simultaneously
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        ids[i] = i + 1;
        pthread_create(&customers[i], NULL, customer_thread, &ids[i]);
        usleep(100000); // 0.1s gap to stagger slightly
    }

    // join all
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        pthread_join(customers[i], NULL);
    }
}
```

```
GNU nano 2.2 hotel_booking.c
return found;
}

void release_room(int room_idx) {
    rooms[room_idx] = -1;
}

void* customer_thread(void* arg) {
    int id = *(int*)arg;
    char tbuf[16];

    // random delay before first attempt (1-3s)
    int initial_wait = (rand() % 3) + 1;
    sleep(initial_wait);

    for (int attempt = 1; attempt ≤ MAX_TRIES; attempt++) {
        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&rooms_mutex);
        printf("[%s] ★ Customer-%d mencoba booking... (percobaan %d)\n", tbuf, id, attempt);
        pthread_mutex_unlock(&rooms_mutex);

        // Try to get a slot (decrement semaphore). If no room, sem_trywait couldn't
        // but we want to block only if willing to wait; here we use sem_trywait
        if (sem_trywait(&rooms_sem) == 0) {
            // got a "permit" - now lock and choose a room index
            pthread_mutex_lock(&rooms_mutex);
            int room = try_book_room(id);
            if (room ≥ 0) {
                timestr(tbuf, sizeof(tbuf));
                int rooms_left = 0;
                // count rooms left for reporting
                for (int i = 0; i < TOTAL_ROOMS; i++) if (rooms[i] == -1) rooms_left++;
                printf("[%s] ✓ Customer-%d berhasil booking! (Kamar %d) - (%d kamar tersisa)\n", tbuf, id, room + 1, rooms_left);
                pthread_mutex_unlock(&rooms_mutex);

                // stay 5s
                sleep(5);
            } else {
                // no room available, release permit and try again
                sem_post(&rooms_sem);
            }
        } else {
            // no permit, try again later
            sleep(1);
        }
    }
}
```

```
GNU nano 2.2 hotel_booking.c
timestr(tbuf, sizeof(tbuf));
printf("[%s] ✖ Customer-%d menyerah setelah %d percobaan\n", tbuf, id, attempt);
pthread_mutex_unlock(&rooms_mutex);
}
}

return NULL;
}

int main() {
    srand(time(NULL));
    // init rooms
    for (int i = 0; i < TOTAL_ROOMS; i++) rooms[i] = -1;

    // init sync primitives
    sem_init(&rooms_sem, 0, TOTAL_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);

    pthread_t customers[TOTAL_CUSTOMERS];
    int ids[TOTAL_CUSTOMERS];

    printf("≡ SISTEM BOOKING HOTEL ≡\n");
    printf("Total Kamar: %d\n", TOTAL_ROOMS);

    // create customer threads almost simultaneously
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        ids[i] = i + 1;
        pthread_create(&customers[i], NULL, customer_thread, &ids[i]);
        usleep(100000); // 0.1s gap to stagger slightly
    }

    // join all
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        pthread_join(customers[i], NULL);
    }
}
```

```

}

int main() {
    srand(time(NULL));
    // init rooms
    for (int i = 0; i < TOTAL_ROOMS; i++) rooms[i] = -1;

    // init sync primitives
    sem_init(&rooms_sem, 0, TOTAL_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);

    pthread_t customers[TOTAL_CUSTOMERS];
    int ids[TOTAL_CUSTOMERS];

    printf("== SISTEM BOOKING HOTEL ==\n");
    printf("Total Kamar: %d\n\n", TOTAL_ROOMS);

    // create customer threads almost simultaneously
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        ids[i] = i + 1;
        pthread_create(&customers[i], NULL, customer_thread, &ids[i]);
        usleep(100000); // 0.1s gap to stagger slightly
    }

    // join all
    for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
        pthread_join(customers[i], NULL);
    }

    // cleanup
    sem_destroy(&rooms_sem);
    pthread_mutex_destroy(&rooms_mutex);

    printf("\nSemua customer selesai.\n");
    return 0;
}

```

ini adalah kode source code nya , dan di bawah ini adalah hasil output dari hasil kode di atas.

```

PS1=faby@fabyar-biee:~/praktikum-sinkronisasiOSP5/tugas-ganjil$ gcc hotel_booking.c -o hotel_booking -pthread
PS1=faby@fabyar-biee:~/praktikum-sinkronisasiOSP5/tugas-ganjil$ ./hotel_booking
== SISTEM BOOKING HOTEL ==
Total Kamar: 5

[12:37:02] ✨ Customer-1 mencoba booking... (percobaan 1)
[12:37:02] ✨ Customer-1 berhasil booking! (Kamar 1) - (4 kamar tersisa)
[12:37:02] ✨ Customer-2 mencoba booking... (percobaan 1)
[12:37:02] ✨ Customer-2 berhasil booking! (Kamar 2) - (3 kamar tersisa)
[12:37:02] ✨ Customer-5 mencoba booking... (percobaan 1)
[12:37:02] ✨ Customer-5 berhasil booking! (Kamar 3) - (2 kamar tersisa)
[12:37:02] ✨ Customer-6 mencoba booking... (percobaan 1)
[12:37:02] ✨ Customer-6 berhasil booking! (Kamar 4) - (1 kamar tersisa)
[12:37:03] ✨ Customer-8 mencoba booking... (percobaan 1)
[12:37:03] ✨ Customer-8 berhasil booking! (Kamar 5) - (0 kamar tersisa)
[12:37:03] ✨ Customer-4 mencoba booking... (percobaan 1)
[12:37:03] ✖ Customer-4 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:04] ✨ Customer-3 mencoba booking... (percobaan 1)
[12:37:04] ✖ Customer-3 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:05] ✨ Customer-7 mencoba booking... (percobaan 1)
[12:37:05] ✖ Customer-7 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:05] ✨ Customer-9 mencoba booking... (percobaan 1)
[12:37:05] ✖ Customer-9 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:05] ✨ Customer-10 mencoba booking... (percobaan 1)
[12:37:05] ✖ Customer-10 gagal booking (kamar penuh), menunggu dan coba lagi...

```

```

[12:37:05] ✖ Customer-10 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:05] ✨ Customer-4 mencoba booking... (percobaan 2)
[12:37:05] ✖ Customer-4 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:06] ✨ Customer-3 mencoba booking... (percobaan 2)
[12:37:06] ✖ Customer-3 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:07] ✨ Customer-7 mencoba booking... (percobaan 2)
[12:37:07] ✖ Customer-7 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:07] ✨ Customer-9 mencoba booking... (percobaan 2)
[12:37:07] ✖ Customer-9 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:07] ✨ Customer-10 mencoba booking... (percobaan 2)
[12:37:07] ✖ Customer-10 gagal booking (kamar penuh), menunggu dan coba lagi...
[12:37:07] 🟡 Customer-1 check-out (Kamar 1) - (1 kamar tersisa)
[12:37:07] 🟡 Customer-2 check-out (Kamar 2) - (2 kamar tersisa)
[12:37:07] ✨ Customer-4 mencoba booking... (percobaan 3)
[12:37:07] ✨ Customer-4 berhasil booking! (Kamar 1) - (1 kamar tersisa)
[12:37:07] 🟡 Customer-5 check-out (Kamar 3) - (2 kamar tersisa)
[12:37:07] 🟡 Customer-6 check-out (Kamar 4) - (3 kamar tersisa)
[12:37:07] 🟡 Customer-8 check-out (Kamar 5) - (4 kamar tersisa)
[12:37:08] ✨ Customer-3 mencoba booking... (percobaan 3)
[12:37:08] ✨ Customer-3 berhasil booking! (Kamar 2) - (3 kamar tersisa)
[12:37:08] ✨ Customer-7 mencoba booking... (percobaan 3)
[12:37:08] ✨ Customer-7 berhasil booking! (Kamar 3) - (2 kamar tersisa)
[12:37:08] ✨ Customer-9 mencoba booking... (percobaan 3)
[12:37:08] ✨ Customer-9 berhasil booking! (Kamar 4) - (1 kamar tersisa)
[12:37:08] ✨ Customer-10 mencoba booking... (percobaan 3)
[12:37:08] ✨ Customer-10 berhasil booking! (Kamar 5) - (0 kamar tersisa)
[12:37:12] 🟡 Customer-4 check-out (Kamar 1) - (1 kamar tersisa)
[12:37:13] 🟡 Customer-3 check-out (Kamar 2) - (2 kamar tersisa)
[12:37:13] 🟡 Customer-7 check-out (Kamar 3) - (3 kamar tersisa)
[12:37:13] 🟡 Customer-9 check-out (Kamar 4) - (4 kamar tersisa)
[12:37:13] 🟡 Customer-10 check-out (Kamar 5) - (5 kamar tersisa)

Semua customer selesai.

```

1.4 Analisis Hasil

A. Apakah Program Berjalan Sesuai Ekspetasi

Program berjalan sesuai dengan yang diharapkan. Simulasi menunjukkan bahwa hanya 5 customer yang dapat menginap bersamaan, sesuai jumlah kamar yang disediakan. Ketika semua kamar penuh, customer lain akan menunggu beberapa detik dan mencoba lagi. Tidak ada dua customer yang menempati kamar yang sama pada waktu bersamaan, sehingga mekanisme sinkronisasi bekerja dengan baik.

B. Masalah yang di Temui Dan Solusinya

- Masalah: Beberapa output saling tumpang tindih ketika beberapa thread mencetak secara bersamaan. Solusi: Menambahkan `pthread_mutex_lock()` dan `pthread_mutex_unlock()` di sekitar proses `printf` agar hanya satu thread yang bisa menulis ke terminal pada satu waktu.
- Masalah: Customer menunggu terlalu lama jika menggunakan `sem_wait()` (blocking). Solusi: Mengganti dengan `sem_trywait()` supaya customer bisa gagal, menunggu 2 detik, dan mencoba lagi sesuai aturan percobaan maksimal 3 kali.
- Masalah: Kadang urutan customer booking tidak berurutan. Solusi: Ini bukan error, tetapi perilaku normal dari multithreading karena setiap thread berjalan paralel. Program sudah benar selama tidak terjadi tumpang tindih kamar.

C. Performa Program

Program berjalan stabil dan efisien. Semua thread dieksekusi paralel dengan respon cepat dan penggunaan CPU yang ringan. Tidak ada tanda *deadlock* atau *resource starvation*. Semaphore dan mutex bekerja optimal menjaga kestabilan akses sumber daya. Waktu eksekusi keseluruhan bergantung pada jumlah thread dan waktu tidur (sleep) yang diberikan.

D. Pelajaran Yang Didapat

Dari praktikum ini saya belajar bahwa:

- Sinkronisasi sangat penting dalam sistem multithreading untuk mencegah konflik data.

- Semaphore efektif digunakan untuk membatasi jumlah akses terhadap sumber daya terbatas.
- Mutex penting untuk menjaga agar operasi yang kritis hanya dijalankan oleh satu thread pada satu waktu.
- Dengan kombinasi kedua mekanisme ini, program dapat berjalan tertib, aman, dan konsisten meskipun banyak thread berjalan bersamaan.

2. Kesimpulan

A. Rangkuman Apa Yang Dipelajari

Dalam praktikum ini saya mempelajari bagaimana cara mengatur kerja beberapa *thread* agar dapat berbagi sumber daya dengan aman melalui mekanisme sinkronisasi. Dengan menggunakan semaphore untuk membatasi jumlah kamar yang dapat digunakan dan mutex untuk melindungi data serta output, program dapat berjalan tanpa konflik, *race condition*, atau deadlock. Konsep ini menggambarkan pentingnya koordinasi antarproses dalam sistem operasi agar hasil eksekusi tetap konsiste

B. Tantangan yang Dihadapi

Tantangan utama adalah memastikan tidak terjadi *race condition* dan menjaga agar output dari setiap thread tidak saling tumpang tindih. Selain itu, memahami perbedaan antara penggunaan `sem_wait()` dan `sem_trywait()` juga cukup menantang karena memengaruhi perilaku blok dan *retry* pada thread. Namun dengan penerapan kombinasi mutex dan semaphore, masalah tersebut dapat diatasi dengan baik.

C. Saran Pengembangan

Ke depan, program dapat dikembangkan dengan:

1. Menambahkan antarmuka yang menampilkan status kamar secara real-time.
2. Menggunakan waktu menginap acak untuk mensimulasikan kondisi yang lebih realistis.
3. Menerapkan prioritas customer (misalnya VIP mendapat prioritas booking).
4. Mencatat log aktivitas ke file agar hasil simulasi dapat dianalisis lebih lanjut.

3. Referensi

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th Edition). Wiley.

Robbins, K. A., & Robbins, S. (2003). *Unix Systems Programming: Communication, Concurrency, and Threads*. Prentice Hal.